

ADR-001 Agente de Desarrollo AI-Native - NexaPlay

ID	Estado	Fecha
ADR-001	Propuesto	20 de Mayo 2026

1. Contexto

NexaPlay es una plataforma de administración multi-tenant que permite a 20 operadoras ofrecer servicios digitales en 6 países de LATAM. Cada operadora tiene configuraciones independientes: parámetros operativos, reglas de negocio por país y límites de operación. La plataforma procesa hasta 200.000 eventos por segundo en tiempo real.

Problema de negocio: El equipo del cliente no puede mantener el ritmo de cambios en el backoffice de forma manual. Un PM necesita poder expresar un requerimiento en lenguaje natural y obtener una acción ejecutada, código generado y un registro auditable del cambio, sin depender de un ciclo de desarrollo tradicional.

Solución: Un agente AI-Native que reciba requerimientos en lenguaje natural, planifique, ejecute acciones reales sobre la API de NexaPlay vía MCP, genere código funcional y entregue salidas verificables.

Núcleo de razonamiento: claude-sonnet 4, ejecución local en el dispositivo del operador.

2. Modelo de Agente

Modelo	Fortalezas	Debilidades	Decisión
ReAct puro	Simple, trazable, bajo overhead	Sin planificación previa; falla en tareas con dependencias de datos entre pasos	Descartado
Plan & Execute	Plan estructurado antes de actuar	Plan estático; no adapta cuando la API responde diferente a lo esperado	Descartado
Multi-agente	Especialización paralela	Coordinación costosa, depuración exponencialmente difícil, tokens duplicados, sin beneficio para un solo objetivo	Descartado
Híbrido ReAct + Plan	Planificación estructurada + ejecución adaptativa	Levemente más complejo que ReAct puro	Elegido

Decisión: Híbrido ReAct + Plan & Execute (agente único)

La elección se justifica con el flujo concreto del problema: el agente primero debe leer el valor de “operational_limit” y sólo entonces decidir si ejecuta el POST. ReAct puro no garantiza ese orden ya que puede intentar escribir antes de haber procesado la lectura. Plan & Execute puro planifica los dos pasos por adelantado, pero si el GET responde que el límite ya está correcto, el POST del plan nunca debería ejecutarse y un plan estático no sabe eso hasta que ya es tarde.

El enfoque híbrido resuelve ambos problemas: el plan declara explícitamente que el POST depende del valor leído, y el loop adaptativo omite el paso si la condición no se cumple. Es el mínimo de complejidad que resuelve el problema correctamente.

Tradeoff aceptado: +2–4s de latencia por la fase de planificación. Correctitud y auditabilidad sobre velocidad en un sistema que modifica configuración de producción.

Responsabilidades por componente de la solución

Componente	Responsabilidad	Tecnología
Orquestador	Ciclo de vida, despacho de fases, coordinación de outputs	Python async
Motor de Planificación	Interpreta requerimiento, produce plan JSON con pasos y precondiciones	Claude (vía ModelRouter)
ReAct Loop	Thought→Action→Observation, control de iteraciones y replanificación	Python event loop
Gestor de Contexto	Token budget por ciclo, ventana deslizante (últimas 3 obs), caché de plan y GET	Python + tiktoken
ModelRouter	Selecciona el modelo Claude correcto según tipo de tarea. Configurable via .env sin tocar código.	Python + .env
MCP Dispatcher	Valida schema, verifica permisos por tier, aplica timeout 5s	Python MCP client
nexaplay_api_call	HTTP real con retry progresivo, idempotency-key por job-id, clasificación 4xx/5xx/red	httpx
code_generator	Código Python + test unitario usando schema real del GET (no campos inventados)	Claude (vía ModelRouter)
Validador de Output	ast.parse del código, check de resumen, verificación SILENT_WRITE_FAILURE. Sin LLM.	ast + Python
Observabilidad	Trazas OTel por paso, JSON logs por acción, métricas al cierre. Local→archivo. OCI→Loki/Grafana sin cambiar código.	OpenTelemetry SDK

3. Diseño de Skills MCP

El agente expone dos skills MCP implementadas como servidor stdio local.

3.1 nexaplay_api_call (obligatoria)

Campo	Especificación
Input	{ endpoint: string, method: "GET" "POST", params?: object, body?: object }
Output	{ success: boolean, data?: any, error?: string, retries_used: number }
Timeout	5 segundos por intento
Reintentos	Máx 3, backoff progresivo: 1s → 2s → 4s
Idempotencia	POST incluye Idempotency-Key: {job_id}-{step}. Scope: por job, no por requerimiento.
Latencia esperada	200–800ms contra el mock local

Clasificación de errores

Tipo de error	Acción	Razón
Error de red / timeout	Reintentar máx 3	Falla transitoria, puede resolverse
HTTP 4xx	No reintentar. Reportar VALIDATION_ERROR al agente	Reintentar produce el mismo resultado
HTTP 5xx	Reintentar máx 3	Falla del servidor, posiblemente transitoria
2xx con body inconsistente	Reportar SILENT_WRITE_FAILURE. Escalar. No reintentar.	El más peligroso: el servidor confirma pero no aplica el cambio

Endpoints cubiertos

- GET /services/{id}/config → lectura de configuración operativa
- POST /services/{id}/config → actualización de parámetros (con idempotency-key)
- GET /events/stream → stream continuo de eventos (chunked, 10 eventos/seg en mock)

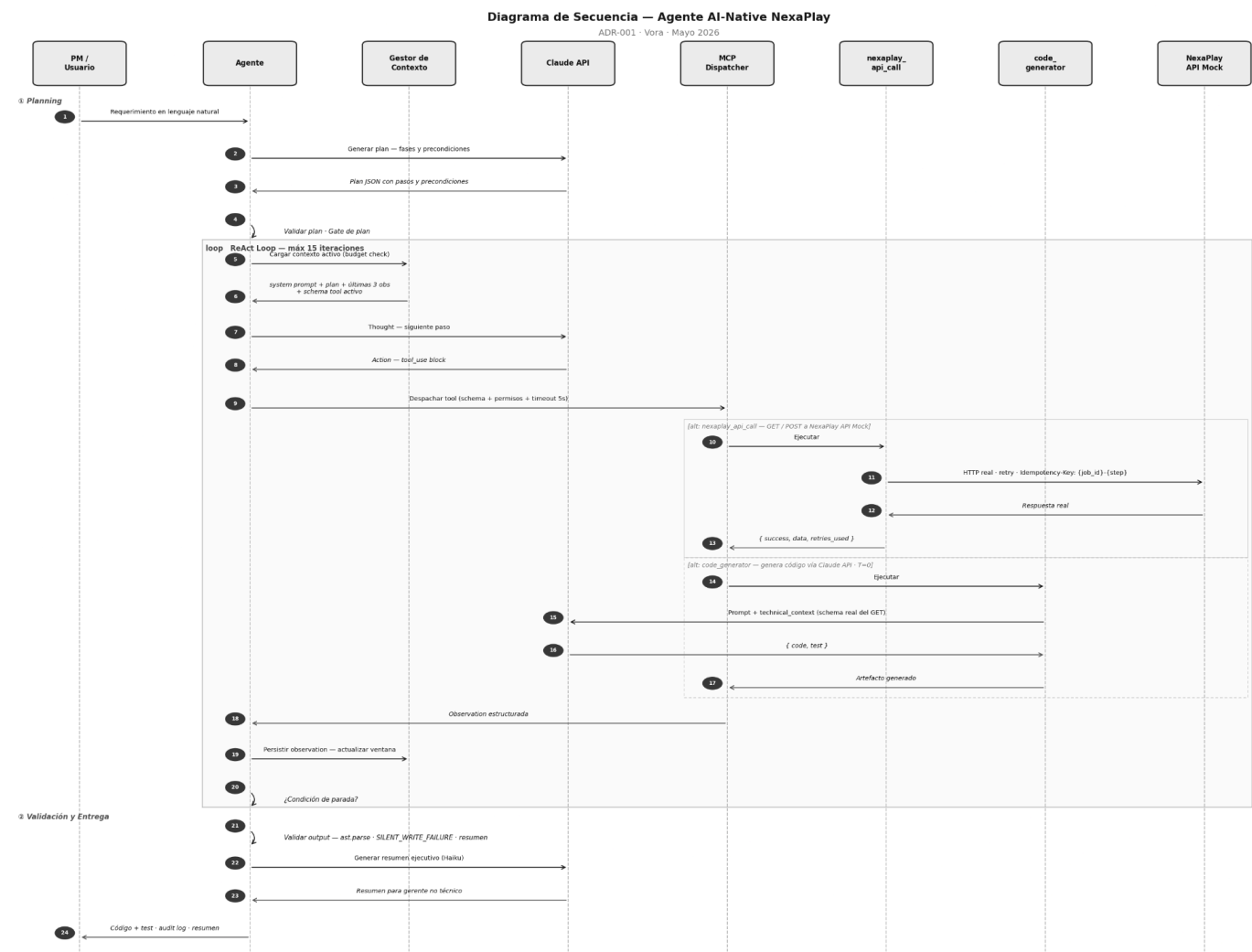
3.2 code_generator (skill a elección — Opción A)

Justificación de la elección: Es la skill con mayor visibilidad en el demo (el evaluador ve código real generado), corresponde al paso 6 del flujo mínimo requerido, y demuestra que el agente razona sobre el contexto técnico antes de producir un artefacto. file_writer y spec_reader son utilitarios de soporte; code_generator es el producto central del agente.

Campo	Especificación
Input	{ requirement: string, technical_context: string, language: "python" "typescript" }
Output	{ code: string, test: string, filename: string, summary: string }
Mecanismo	Llamada a Claude API con prompt especializado, temperatura 0. El código NO se ejecuta automáticamente.
Latencia esperada	3–8s (llamada a Claude API)
Idempotencia	Temperatura 0 = mismo input → mismo output. Determinístico.
Error: GENERATION_FAILED	Claude API no responde o respuesta malformada
Error: INVALID_CONTEXT	Contexto técnico vacío — el agente no puede generar código sin la estructura de la API

Dependencia crítica entre skills: code_generator siempre se invoca después de nexaplay_api_call. El technical_context que recibe es el schema real de la respuesta del GET — garantizando que el código use los campos que la API realmente devuelve y no campos inventados.

4. Orquestación



5. Manejo de Contexto

Composición del contexto activo por ciclo ReAct

Componente	Tokens estimados	Política
System prompt (fijo)	~1.500	Cargado una vez, inmutable durante el job
Plan actual (JSON)	~800–1.500	En contexto hasta completar el job
Últimas 3 observaciones	~2.000–3.000	Ventana deslizante; anteriores → resumidas con Haiku
Schema del tool activo	~500–800	Solo el tool del paso actual (carga selectiva)
Prompt de thought	~300	Generado por el runtime por ciclo
Total por ciclo (input)	~5.100–7.100	—
Respuesta Claude (output)	~500–1.500	Thought + action block
Total por paso ReAct	~6.000–8.600	—

Ejecución (10 pasos): ~70.000–85.000 tokens ≈ \$0.25–\$0.30

- **Contexto activo:** System prompt + plan + últimas 3 observaciones, schema del tool actual.
- **Resumido con Haiku:** Observaciones anteriores a las últimas 3 cuando el buffer se llena, es 10x más barato que Sonnet para tarea mecánica de compresión.
- **Persistido en disco:** Log completo, respuestas completas de la API, código generado, timestamps.
- **Cache:** Respuestas GET en un dict python y plan cacheado por hash del requerimiento

6. Gestión de Errores

Escenario	Manejo	Recuperación
Error de red / timeout	Reintentar 3x con backoff 1s→2s→4s	Si agota: NETWORK_ERROR como observation
HTTP 4xx	No reintentar. VALIDATION_ERROR al agente con descripción	Agente evalúa si puede corregir el body; si no, escala
HTTP 5xx	Reintentar 3x	Si agota: SERVER_ERROR, escalar con traza
2xx con body incorrecto	SILENT_WRITE_FAILURE — el cambio no se aplicó aunque la API dijo OK	Escalar inmediatamente. No reintentar. Es el fallo más peligroso en multi-tenant.
Plan inválido	Validación estructural al generarlo	Regenerar una vez; si falla de nuevo: rechazar
Loop detectado	Mismo tool + mismos args × 3 consecutivas	Abortar, guardar estado parcial, notificar
Límite de iteraciones	Hard stop a las 15 iteraciones	Guardar estado parcial, notificar con lo completado
Código no parseable	ast.parse falla en el Validador de Output	1 reintentado pasando el error como observation. Si falla: entregar marcado UNVALIDATED — honesto es mejor que silencioso.

Rollback: El agente es workspace-scoped. El código nunca se despliega automáticamente. El POST es la única acción con efecto externo y se logea con body completo, timestamp y job-id antes de ejecutarse.

7. Seguridad y Límites

AUTÓNOMO — el agente actúa sin aprobación

- ✓ GET /services/{id}/config — lectura, sin efectos de estado, GET /events/stream — lectura, sin efectos de estado, Generar código (artefacto local, no se ejecuta), Escribir archivos en ./workspace/{job_id}/

REQUIERE CONFIRMACIÓN EXPLÍCITA

POST /services/{id}/config — el agente muestra el body antes de ejecutar; el usuario debe escribir "confirmar" para proceder

BLOQUEADO — hard stop sin excepción

✗ Ejecutar código generado sin revisión humana, escribir fuera de `./workspace/{job_id}/`, llamar endpoints no documentados en la spec provista, leer variables de entorno o secrets del sistema operativo, ejecución de shell arbitrario, el agente no puede llamarse a sí mismo (sin recursión)

Secrets y Prompt Injection

Aspecto	Implementación
Secrets	API key de Claude y credenciales desde <code>.env</code> . Nunca aparecen en el contexto de Claude, en los logs ni en el código generado. (<code>env.example</code> documenta todas las variables)
Prompt injection	Los requerimientos del usuario son input no confiable. Se encapsulan en <code><user_requirement>...</user_requirement></code> . Las respuestas de la API son datos, no instrucciones. System prompt es inmutable e inyectado por el runtime.
Validación de rutas	El workspace bloquea <code>../</code> y symlinks antes de cualquier escritura de archivo.

8. Decisiones de Costo

Tipo de solicitud	Pasos	Tokens est.	Costo est.
Solo consulta (GET sin cambio)	3–5	~30.000	~\$0.09
Flujo completo (GET + eval + POST + código)	8–12	~75.000–100.000	~\$0.23–\$0.30
Con replanificación (+5 pasos)	+5	+50.000	+\$0.15
Resúmenes Haiku (2–4 llamadas)	—	~20.000	~\$0.01
Solicitud típica completa	—	~90.000	~\$0.27

Referencia: *Sonnet 4* ≈ \$3/MTok input · \$15/MTok output. *Haiku 4.5* ≈ \$0.25/MTok input.

Estrategias de optimización

Estrategia	Implementación	Ahorro
Carga selectiva de schemas	Solo el schema del tool del paso actual	20–30% menos tokens de entrada
Cache de GET (in-memory)	Dict Python con TTL 60s	Elimina llamadas redundantes en reintentos
Haiku para resúmenes	Observaciones >2K tokens resumidas con Haiku	~10x más barato por llamada de resumen
Temperatura = 0	Todas las llamadas de producción	Reproducibilidad; reduce retries por variabilidad
Cache de plan	Hash del requerimiento → reutilizar plan	Elimina llamada de planificación en reintentos del demo